

SPRING BOOT & MICROSERVICES

Support de cours — Construire des applications Java modernes
et des architectures distribuées

Niveau : Débutant / Intermédiaire

Prérequis : bases du langage Java, notions de programmation orientée objet

8 pages — Document pédagogique

1. Introduction à Spring Boot

Spring Boot est un framework Java qui simplifie considérablement le développement d'applications basées sur **Spring**. Il propose une configuration par défaut intelligente ("convention plutôt que configuration"), un serveur embarqué et une gestion simplifiée des dépendances, permettant de démarrer un projet fonctionnel en quelques minutes.

Définition clé

Spring Boot repose sur le principe de l'auto-configuration : il détecte les bibliothèques présentes dans le projet et configure automatiquement les composants nécessaires, sans fichiers XML fastidieux.

Pourquoi Spring Boot ?

- **Serveur embarqué** — Tomcat ou Netty intégré, pas besoin de serveur externe
- **Starters** — dépendances prêtes à l'emploi (spring-boot-starter-web, etc.)
- **Auto-configuration** — réduit fortement le code de configuration manuel
- **Production-ready** — métriques, health-checks et logs via Actuator

Créer un projet avec Spring Initializr

Le moyen le plus rapide de démarrer un projet est d'utiliser **start.spring.io**, qui génère un squelette de projet Maven ou Gradle avec les dépendances choisies (Web, JPA, Security, etc.).

Structure d'un projet Spring Boot

```
mon-projet/  
|- src/main/java/com/exemple/app/  
|   |- Application.java  
|   |- controller/  
|   |- service/  
|   |- repository/  
|   `-- model/  
|- src/main/resources/  
|   `-- application.properties  
`-- pom.xml
```

2. Créer une première application

Une application Spring Boot démarre à partir d'une classe principale annotée avec **@SpringBootApplication**, qui combine trois annotations : **@Configuration**, **@EnableAutoConfiguration** et **@ComponentScan**.

Classe principale

```
package com.exemple.app;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class Application {
    public static void main(String[] args) {
        SpringApplication.run(Application.class, args);
    }
}
```

Premier contrôleur REST

```
package com.exemple.app.controller;

import org.springframework.web.bind.annotation.*;

@RestController
@RequestMapping("/api/hello")
public class HelloController {

    @GetMapping
    public String direBonjour() {
        return "Bonjour depuis Spring Boot !";
    }
}
```

Annotations essentielles

Annotation	Rôle
@RestController	Déclare un contrôleur exposant des endpoints REST
@Service	Marque une classe contenant la logique métier
@Repository	Marque une classe d'accès aux données
@Autowired	Injecte automatiquement une dépendance
@GetMapping / @PostMapping	Associe une méthode à une route HTTP

3. Injection de dépendances et architecture en couches

Spring repose sur l'**inversion de contrôle** (IoC) : plutôt que de créer lui-même ses dépendances, un objet les reçoit du conteneur Spring, ce qui facilite les tests et le découplage du code.

Architecture en couches classique

- **Controller** — reçoit les requêtes HTTP et retourne les réponses
- **Service** — contient la logique métier de l'application
- **Repository** — gère l'accès et la persistance des données
- **Model / Entity** — représente les objets métier et les tables de données

Exemple d'injection par constructeur

```
@Service
public class UtilisateurService {

    private final UtilisateurRepository repository;

    public UtilisateurService(UtilisateurRepository repository) {
        this.repository = repository;
    }

    public List<Utilisateur> listerTous() {
        return repository.findAll();
    }
}
```

Bonne pratique

Préférer l'injection par constructeur à l'injection par champ (`@Autowired` sur un attribut) : elle rend les dépendances explicites et facilite l'écriture de tests unitaires.

Persistance avec Spring Data JPA

```
public interface UtilisateurRepository
    extends JpaRepository<Utilisateur, Long> {
    Optional<Utilisateur> findByEmail(String email);
}
```

4. Construire une API REST complète

Spring Boot facilite la création d'API REST respectant les conventions HTTP : chaque méthode du contrôleur correspond à une action sur une ressource.

Contrôleur CRUD complet

```
@RestController
@RequestMapping("/api/produits")
public class ProduitController {

    private final ProduitService service;

    public ProduitController(ProduitService service) {
        this.service = service;
    }

    @GetMapping
    public List<Produit> lister() {
        return service.listerTous();
    }

    @GetMapping("/{id}")
    public Produit obtenir(@PathVariable Long id) {
        return service.trouverParId(id);
    }

    @PostMapping
    public Produit creer(@RequestBody Produit produit) {
        return service.creer(produit);
    }

    @PutMapping("/{id}")
    public Produit modifier(@PathVariable Long id, @RequestBody Produit produit) {
        return service.modifier(id, produit);
    }

    @DeleteMapping("/{id}")
    public void supprimer(@PathVariable Long id) {
        service.supprimer(id);
    }
}
```

Codes HTTP courants

- 200 OK — requête traitée avec succès
- 201 Created — ressource créée avec succès
- 400 Bad Request — requête invalide
- 404 Not Found — ressource introuvable
- 500 Internal Server Error — erreur côté serveur

5. Introduction aux microservices

L'architecture **microservices** consiste à découper une application en plusieurs services indépendants, chacun responsable d'une fonctionnalité métier précise, déployable et évolutif séparément.

Monolithe vs microservices

Critère	Monolithe	Microservices
Déploiement	Une seule unité	Indépendant par service
Mise à l'échelle	Application entière	Service par service
Technologie	Unique	Peut varier par service
Complexité	Simple au départ	Complexité distribuée

Avantages et défis

- **Avantages** — déploiements indépendants, résilience, choix technologique flexible
- **Défis** — communication réseau, cohérence des données, supervision distribuée

Communication entre services

Les microservices communiquent généralement via des appels **REST/HTTP** synchrones ou via des **files de messages** asynchrones (Kafka, RabbitMQ) pour découpler davantage les services.

6. Spring Cloud : l'écosystème microservices

Spring Cloud fournit des outils prêts à l'emploi pour résoudre les problématiques communes des architectures distribuées : découverte de services, configuration centralisée, résilience et routage.

Composants clés

- **Eureka** — annuaire de services permettant leur découverte dynamique
- **Spring Cloud Gateway** — passerelle API centralisant le routage des requêtes
- **Spring Cloud Config** — configuration centralisée et externalisée
- **Resilience4j** — gestion des pannes via circuit breakers et retries
- **OpenFeign** — client HTTP déclaratif pour appeler d'autres services

Exemple : client Feign

```
@FeignClient(name = "service-commandes")
public interface CommandeClient {

    @GetMapping("/api/commandes/{id}")
    Commande obtenirCommande(@PathVariable Long id);
}
```

Exemple : enregistrement Eureka

```
# application.properties
spring.application.name=service-produits
eureka.client.service-url.defaultZone=http://localhost:8761/eureka/
```

À retenir

Spring Cloud ne remplace pas Kubernetes : il opère au niveau applicatif (Java) tandis que Kubernetes orchestre au niveau infrastructure. Les deux peuvent être combinés.

7. Résilience, sécurité et configuration

Circuit Breaker avec Resilience4j

Un **circuit breaker** empêche les appels répétés vers un service défaillant, évitant une cascade de pannes dans l'architecture distribuée.

```
@CircuitBreaker(name = "serviceCommandes", fallbackMethod = "solutionDeSecours")
public Commande obtenirCommande(Long id) {
    return commandeClient.obtenirCommande(id);
}

public Commande solutionDeSecours(Long id, Throwable ex) {
    return new Commande(id, "Service indisponible");
}
```

Sécuriser une API avec Spring Security

Spring Security gère l'authentification et les autorisations, souvent via des jetons **JWT** dans les architectures microservices sans état.

```
@Configuration
@EnableWebSecurity
public class SecurityConfig {

    @Bean
    public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
        http.authorizeHttpRequests(auth -> auth
            .requestMatchers("/api/public/**").permitAll()
            .anyRequest().authenticated()
        )
        .oauth2ResourceServer(oauth2 -> oauth2.jwt(withDefaults()));
        return http.build();
    }
}
```

Configuration externalisée

Les paramètres (URL de base de données, ports, clés) sont définis dans **application.properties** ou **application.yml**, et peuvent varier selon les profils (dev, test, prod) via **spring.profiles.active**.

8. Déploiement et conclusion

Conteneuriser une application Spring Boot

```
FROM eclipse-temurin:21-jre-alpine
WORKDIR /app
COPY target/mon-service-0.0.1.jar app.jar
EXPOSE 8080
ENTRYPOINT ["java", "-jar", "app.jar"]
```

Observabilité avec Spring Boot Actuator

Le module **Actuator** expose des endpoints prêts à l'emploi pour surveiller l'application en production : santé, métriques, informations de version.

```
management.endpoints.web.exposure.include=health,info,metrics

# GET /actuator/health -> { "status": "UP" }
```

Points clés à retenir

- Spring Boot simplifie la configuration grâce à l'auto-configuration
- L'architecture en couches (Controller / Service / Repository) structure le code
- L'injection de dépendances favorise le découplage et la testabilité
- Les microservices découpent une application en services indépendants
- Spring Cloud fournit découverte, routage et résilience pour les architectures distribuées
- Actuator et les conteneurs facilitent la supervision et le déploiement

Pour aller plus loin

- Documentation officielle Spring (spring.io)
- Tests avec Spring Boot Test et JUnit 5
- Messagerie asynchrone avec Kafka ou RabbitMQ
- Déploiement sur Kubernetes avec Helm

Conclusion

Spring Boot offre une base solide et productive pour construire des applications Java modernes, tandis que Spring Cloud permet de faire évoluer ces applications vers des architectures microservices robustes. La pratique sur des projets réels reste la meilleure façon de consolider ces notions.